AURA - Stage 3 Verification Engine

Implementation and acceptance report | 24 October 2026 | Controlled local prototype

Engineering milestone completed: AURA can now check whether every declared outcome, side-effect condition, and mandatory process requirement is supported by independently collected evidence. It returns **PASS, FAIL, or UNCERTAIN** with an auditable certificate.

ACCEPTANCE MEASURE	RESULT
Automated tests	76 passed (35 Stage 2 + 41 new Stage 3)
Stage 1 authored verification cases	12/12 expected verdicts and decision statuses
Fresh local sandbox runs	6/6 expected task-completion decisions
Outputs	Structured contract certificate, evidence references, unresolved facts, measured costs
Research claims	No external benchmark, learned agent comparison or SOTA superiority claim

The essential idea

A tool can return a success message while the actual database, report, or side effects are incorrect. AURA checks the **complete task contract**, not the tool-success message. It also distinguishes failure from a lack of independent evidence.

Stage 3 architecture

LAYER	ROLE
Stage 1 contract	Defines outcomes, permitted changes, required process, observation scope
Stage 2 evidence	Stores trusted SQLite/filesystem snapshots, registered observations, trace and frame receipts
Stage 3 predicate evaluator	Checks six restricted operators, counts, cross-artifact facts, process requirements
Stage 3 verdict arbiter	FAIL dominates ordinary unknown; evidence conflicts force UNCERTAIN
Stage 3 certificate	Per-obligation status, contract hash, scope receipts, evidence refs, capture and probe costs

The implemented evaluator is conservative: it checks atomic predicates and their evidence requirements. It is not a possible-world solver or an adaptive inspection planner.

Stage 1 reference acceptance

AUTHORED REFERENCE CASE	STAGE 3 DECISION
01 Complete success	PASS
02 Missing report	FAIL
03 Unauthorized edit	FAIL
04 Duplicate record	FAIL
05 Unknown notification	UNCERTAIN
06 Known bad + unknown	FAIL
07 Incomplete frame read	UNCERTAIN
08 Contradictory evidence	UNCERTAIN*
09 Missing process validation	FAIL
10 Unobserved transient effect	UNCERTAIN
11 Valid alternative path	PASS
12 Unobserved external frame	UNCERTAIN

*Case 08 additionally returns **decision_status=evidence_conflict**. These are **synthetic authoring fixtures** with pre-authored facts, not captured real-world execution. All 12 match their expected verdict and status.

A necessary fixture compatibility correction

Stage 1 fixture JSON shapes were valid, but **52 synthetic observation digests** did not match Stage 2 canonical hashing. The new test-only fixture copies recompute those hashes. Their facts, task contracts, and expected verdicts were not altered. The original Stage 1 archive remains unchanged; old/new values appear in an audit JSON. This limitation is disclosed rather than treating synthetic receipts as verified state evidence.

Three-valued decision rules

VERDICT	WHEN IT IS JUSTIFIED
PASS	Every contract obligation has supported trustworthy evidence, including sufficient complete-frame receipts
FAIL	At least one trusted fact proves a violation; other missing observations do not hide known failures
UNCERTAIN	A required observation or coverage receipt is unavailable, or trustworthy same-epoch facts conflict

Six new, real local demonstration runs

SCENARIO	VERDICT	REASON
Clean	PASS	all obligations supported
Missing Report	FAIL	cross artifact mismatch, required report absent
Collateral	FAIL	out of scope mutation
Silent Noop	FAIL	cross artifact mismatch, required value mismatch
Denied Readback	UNCERTAIN	required evidence unavailable
Missing Validation	FAIL	process precedence violated

Every local case includes a captured database, workspace, before/after snapshots, read-only observations, a trace, and a Stage 3 certificate. A single reproducibility command regenerates these runs; unlike the synthetic reference cases, Stage 2 independently captures the local state.

Negative tests and integrity checks

- Hash changes and mismatched snapshot references are rejected rather than allowed to certify PASS.
- Same-epoch contradictory independent facts result in UNCERTAIN and an evidence-conflict diagnostic.
- Incomplete frame coverage and transient-history demands cannot be certified by unrelated snapshots.
- Boolean values cannot pass exact integer-count checks; two null values cannot prove report/database equality.
- Self-reported values, wrong epochs, unregistered probes, duplicate observation IDs, and unsupported operators cannot establish completion.

What these results do not prove

The 76 automated tests and 18 controlled acceptance runs are software engineering validation. They do not establish generalization to unfamiliar agents, completeness of arbitrary contracts, an unconditional soundness theorem, external-service behavior, SOTA comparison, or research novelty.

Trust boundaries and known limitations

1. The subject of verification is the **declared contract**, not unstated natural-language user intent.

2. The local sandbox assumes cooperative quiescence. Database and filesystem captures are **not** an atomic snapshot of a live distributed world.

3. Stage 2 tracing covers proxied calls and does not prove true causal derivation or instrument every possible side channel.

4. The frame guarantee is restricted to named, completely inspected surfaces and a restricted grammar of allowed operations.

5. A before/after state diff cannot reveal every temporary reversed write. A complete intermediate journal is not implemented.

6. Hash integrity depends on protected verifier-owned evidence. SHA-256 does not authenticate a compromised host.

7. External provider readbacks and conditional trust are not generally established. Missing evidence leads to UNCERTAIN.

8. The Stage 3 engine does not implement bounded exact possible-world reasoning or CERA adaptive evidence selection.

Reproduce the implementation

```
python -m pip install -e '.[test]'
python -m pytest -q
python run_demo.py --scenario clean --out ./demo_clean
python verify_task.py --run ./demo_clean --out ./demo_clean/certificate.json
python reproduce_stage3.py --out ./stage3_reproduced
```

Use fresh output directories because evidence and certificates are create-once records.

Next engineering stage

Stage 4: Evidence Model and Logical Reasoning. Build the obligation/evidence dependency model and bounded possible-world evaluator. Compare it against this Stage 3 conservative baseline and test observable indistinguishability. Adaptive CERA inspection planning is Stage 5.

GitHub status: No commits or pushes were performed.